

Master Informatique et Télécommunications - MIT

Module : Ingénierie des données

Université Mohammed V de Rabat - UM5

Mini-projet Systèmes NoSQL

# MONGODB - MGO-6

## Plateforme de streaming musical

Modélisation et analyse d'un historique d'écoutes Spotify avec MongoDB

Répertoire GitHub :

<https://github.com/AmraniCh/spotify-listening-analytics-mongodb>

Tableau de bord :

<https://spotify-listening-analytics-app.streamlit.app>

## Sommaire

|                                                                                                                |    |
|----------------------------------------------------------------------------------------------------------------|----|
| Sommaire .....                                                                                                 | 2  |
| 1. Contexte métier .....                                                                                       | 4  |
| 1.1 Le besoin .....                                                                                            | 4  |
| 1.2 Le travail demandé.....                                                                                    | 4  |
| 1.3 Source et volume des données.....                                                                          | 4  |
| 2. Modélisation des données.....                                                                               | 5  |
| 2.1 Le dataset et ses limites.....                                                                             | 5  |
| 2.2 Choix de modélisation : imbrication vs référencement.....                                                  | 5  |
| 2.3 Exemple de document — collection écoutes .....                                                             | 6  |
| 2.4 Exemple de document — collection utilisateurs .....                                                        | 6  |
| 3. Mise en place de l'environnement et import des données.....                                                 | 6  |
| 3.1 Environnement technique .....                                                                              | 6  |
| 3.2 Résultat de l'import .....                                                                                 | 6  |
| 4. Opérations CRUD .....                                                                                       | 8  |
| 4.1 CREATE — Ajouter une nouvelle écoute.....                                                                  | 8  |
| 4.2 READ — Rechercher les écoutes.....                                                                         | 9  |
| 4.3 UPDATE — Modifier les métadonnées d'un morceau .....                                                       | 14 |
| 4.4 DELETE — Supprimer les écoutes d'un utilisateur.....                                                       | 14 |
| 5. Analyses .....                                                                                              | 15 |
| 5.1 Analyses minimales attendues .....                                                                         | 15 |
| 5.1.1 Morceaux les plus écoutés.....                                                                           | 15 |
| 5.1.2 Temps total d'écoute par genre.....                                                                      | 16 |
| 5.1.3 Artistes les plus écoutés.....                                                                           | 17 |
| 5.1.4 Répartition des écoutes par plateforme.....                                                              | 18 |
| 5.1.5 Évolution des écoutes dans le temps.....                                                                 | 19 |
| 5.2 Analyses complémentaires (au moins 3 exigées) .....                                                        | 19 |
| 5.2.1 Genre préféré de chaque utilisateur .....                                                                | 19 |
| 5.2.2 Heures de forte écoute .....                                                                             | 20 |
| 5.2.3 Taux de réécoute des morceaux populaires .....                                                           | 22 |
| 6. Indexation .....                                                                                            | 23 |
| Les requêtes d'indexation ont été regroupées dans le fichier queries/indexation.js, exécuté dans mongosh. .... | 23 |
| 6.1 Avant indexation — plan d'exécution.....                                                                   | 24 |
| 6.2 Création des index.....                                                                                    | 25 |
| 6.3 Comparaison après indexation (index morceau.artiste) .....                                                 | 26 |
| 6.4 Comparaison avant / après .....                                                                            | 27 |
| 7. Résultats obtenus .....                                                                                     | 28 |

|                                                    |    |
|----------------------------------------------------|----|
| 7.1 Import et modèle .....                         | 28 |
| 7.2 Opérations CRUD .....                          | 28 |
| 7.3 Analyses .....                                 | 28 |
| 7.4 Répartition par plateforme .....               | 29 |
| 7.5 Indexation .....                               | 29 |
| 7.6 Un résultat à interpréter avec prudence .....  | 29 |
| 8. Tableau de bord (livrable complémentaire) ..... | 30 |
| 8.1 Stack technique.....                           | 30 |
| 8.2 Contenu.....                                   | 30 |
| 8.3 Déploiement .....                              | 30 |
| 8.4 Capture d'écran.....                           | 31 |
| 9. Analyse critique.....                           | 32 |
| 9.1 Le prix de l'imbrication .....                 | 32 |
| 9.2 L'enrichissement par une source externe.....   | 32 |
| 9.3 La simulation des utilisateurs.....            | 32 |
| 9.4 La stratégie d'indexation.....                 | 32 |
| 9.5 Les limites de MongoDB observées .....         | 33 |

# 1. Contexte métier

## 1.1 Le besoin

Une plateforme de streaming musical souhaite analyser les habitudes d'écoute de ses utilisateurs. L'objectif est de mieux comprendre les préférences musicales, d'identifier les périodes de forte activité et de mesurer la popularité des artistes.

Ces informations servent plusieurs usages métier. Elles permettent d'alimenter un moteur de recommandation, d'adapter les playlists éditoriales aux goûts réels des auditeurs, et de dimensionner l'infrastructure en fonction des pics d'écoute.

## 1.2 Le travail demandé

Le projet consiste à modéliser une collection `ecoutes` contenant les informations relatives à l'utilisateur, au morceau, à l'artiste, au genre musical et à la date d'écoute.

Cinq objectifs sont attendus.

- Concevoir un modèle de données adapté à MongoDB.
- Importer et manipuler un jeu de données réaliste.
- Réaliser les quatre opérations CRUD.
- Écrire les requêtes d'analyse, minimales et complémentaires.
- Mettre en place une stratégie d'indexation et en mesurer l'effet.

## 1.3 Source et volume des données

Le jeu de données est un historique d'écoutes Spotify publié sur Kaggle. Chaque ligne correspond à un événement d'écoute, et non à un morceau. Un même titre peut donc apparaître plusieurs fois.

| Élément            | Valeur                             |
|--------------------|------------------------------------|
| Source             | Kaggle — Spotify Streaming History |
| Volume brut        | 149 860 écoutes                    |
| Période couverte   | 8 juillet 2013 au 15 décembre 2024 |
| Granularité        | une ligne = un événement d'écoute  |
| Artistes distincts | 4 113                              |
| Morceaux distincts | 16 527                             |
| Albums distincts   | 7 948                              |
| Volume retenu      | 12 000 écoutes (voir §2.1)         |

Les colonnes disponibles sont `spotify_track_uri`, `ts`, `platform`, `ms_played`, `track_name`, `artist_name`, `album_name`, `reason_start`, `reason_end`, `shuffle` et `skipped`.

Ce jeu de données présente trois écarts importants avec le besoin exprimé. Ils sont détaillés dans la section suivante.

## 2. Modélisation des données

### 2.1 Le dataset et ses limites

Le jeu de données provient de Kaggle (Spotify Streaming History) : 149 860 lignes, du 8 juillet 2013 au 15 décembre 2024. Chaque ligne est un événement d'écoute, et non un morceau.

Trois problèmes, trois décisions :

| Problème                                                                       | Décision                                                                                                     | Scripts                                               |
|--------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| 149 860 écoutes, alors que l'énoncé en demande 8 000 à 15 000.                 | Échantillon de <b>12 000 écoutes</b> , sur une période continue de juillet 2023 à décembre 2024.             | explore.py<br>sample.py                               |
| Aucune colonne genre, alors que deux analyses exigées en dépendent.            | Enrichissement via l' <b>API Last.fm</b> (tags par artiste). Couverture obtenue : <b>96,7 %</b> des écoutes. | fetch_tags.py<br>build_categories.py<br>map_genres.py |
| Aucune colonne utilisateur : le dataset est l'historique d'une seule personne. | <b>16 utilisateurs simulés</b> , par découpage chronologique (750 écoutes chacun).                           | build_documents.py                                    |

**Volume** : l'échantillonnage est chronologique et non aléatoire. Les écoutes sont très inégalement réparties (26 320 en 2017, 23 en 2014) : un tirage aléatoire aurait produit une série discontinue, privant de sens l'analyse de l'évolution dans le temps.

**Genre** : l'API Spotify a été écartée (abonnement Premium obligatoire depuis février 2026 et suppression des endpoints de métadonnées en masse). Last.fm n'expose pas de champ genre, mais des tags communautaires bruités (seen live, 60s, american). Une liste blanche de 24 genres a donc été dérivée des données elles-mêmes : comptage des tags les plus fréquents sur les 1 113 artistes de l'échantillon, puis sélection manuelle des genres réels.

**Utilisateurs** : la simulation est une limite assumée. Chaque utilisateur n'étant actif que sur une période distincte, les analyses temporelles par utilisateur reflètent un artefact du découpage. Les analyses temporelles globales restent valides.

### 2.2 Choix de modélisation : imbrication vs référencement

- morceau est imbriqué (Embedded) dans chaque document écoutes : titre, artiste, album et genre sont dupliqués dans chaque écoute. Ce choix évite une jointure applicative à chaque lecture et convient à un modèle où le morceau n'est qu'un attribut contextuel de l'écoute.
- utilisateur est référencé (id\_utilisateur) plutôt qu'imbriqué : avec 750 écoutes par utilisateur, dupliquer le profil complet dans chaque document aurait multiplié inutilement le volume de données et compliqué la mise à jour du profil.

Cette dualité illustre le compromis classique embedding vs referencing : l'imbrication optimise la lecture au prix d'une duplication (voir §4.3, updateMany), tandis que le référencement limite la duplication au prix d'un accès en deux temps lorsqu'on a besoin du profil complet.

## 2.3 Exemple de document — collection écoutes

Exemple obtenu via `db.ecoutes.findOne()` :

```
{
  "_id": "eco_00001",
  "id_utilisateur": "user_01",
  "spotify_track_uri": "6BnY0YochAURSXSr3d5N70",
  "morceau": {
    "titre": "Andy, You're A Star",
    "artiste": "The Killers",
    "album": "Hot Fuss",
    "genre": "indie"
  },
  "plateforme": "android",
  "ms_played": 194080,
  "reason_start": "trackdone",
  "reason_end": "trackdone",
  "shuffle": true,
  "skipped": false,
  "date_ecoute": { "$date": "2023-10-05T03:08:51Z" }
}
```

## 2.4 Exemple de document — collection utilisateurs

```
{
  "_id": "user_01",
  "nom": "Emma Laurent",
  "pays": "FR"
}
```

# 3. Mise en place de l'environnement et import des données

## 3.1 Environnement technique

- Environnement virtuel Python (venv) avec les dépendances listées dans `requirements.txt` (pandas, pymongo, dnspython, python-dotenv, requests, streamlit, altair).
- Base MongoDB 7 exécutée dans un conteneur Docker : `docker run -d --name mongo -p 27017:27017 mongo:7`
- Pipeline en sept scripts (détaillés en §3.2) : exploration, échantillonnage, enrichissement des genres, construction des documents et import. L'import est idempotent : les collections sont vidées avant chaque insertion, le script peut donc être rejoué sans risque.

## 3.2 Résultat de l'import

La préparation des données repose sur un pipeline de sept scripts, exécutés séquentiellement. Chaque script lit la sortie du précédent et produit un nouveau fichier : aucun ne modifie les données en place, ce qui permet de rejouer n'importe quelle étape sans casser les autres.

### 3.2.1 Le pipeline

| # | Script              | Rôle                                                                                                                                                                             | Résultat                                     |
|---|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| 1 | explore.py          | Statistiques du jeu de données brut : volume, période, valeurs manquantes, cardinalité, répartition par année et par plateforme.                                                 | reports/exploration.txt                      |
| 2 | sample.py           | Réduit le dataset à 12 000 écoutes sur une période continue, en respectant la fourchette imposée par l'énoncé.                                                                   | sample.csv<br>(12 000 lignes)                |
| 3 | fetch_tags.py       | Interroge l'API Last.fm pour les 1 113 artistes de l'échantillon et récupère leurs tags bruts. Résultats mis en cache : l'API n'est appelée qu'une seule fois par artiste.       | artist_tags.json                             |
| 4 | build_categories.py | Compte la fréquence de tous les tags et affiche les 60 plus courants. Aucun appel API : lit le cache. Sert à construire la liste blanche de genres à partir des données réelles. | Affichage terminal                           |
| 5 | map_genres.py       | Applique la liste blanche : pour chaque artiste, retient le premier tag correspondant à un genre valide, sinon Unknown. Mesure la couverture obtenue.                            | artist_genres.json<br>(96,7 % de couverture) |
| 6 | build_documents.py  | Fusionne l'échantillon et les genres, imbrique le morceau, génère les 16 utilisateurs et type les dates au format Extended JSON.                                                 | listens.json<br>(12 000 documents)           |
| 7 | import_mongodb.py   | Vide puis remplit les collections. Décode l'Extended JSON en véritables objets Date.                                                                                             | Collections ecoutes et utilisateurs          |

```
$ python scripts/build_documents.py
Genres: 25
Unknown genre: 392
Saved to data/processed/listens.json

$ python scripts/import_mongodb.py
ecoutes: 12000
utilisateurs: 16
```

### 3.2.2 Vérification

Vérification effectuée directement dans le shell MongoDB :

```
$ docker exec -it mongo mongosh
> use streaming
> db.ecoutes.countDocuments() // 12000
> db.utilisateurs.countDocuments()
```

```
streaming> db.ecoutes.countDocuments()
12000
```

```
streaming> db.utilisateurs.countDocuments()
16
```

## 4. Opérations CRUD

Les quatre opérations CRUD ont été implémentées dans le fichier `queries/crud.js` et exécutées dans `mongosh` (mode interactif, requête par requête, pour permettre les captures d'écran). Le fichier peut également être rejoué en une seule fois avec : `docker exec -i mongo mongosh streaming < queries/crud.js` (option `-i` seule, sans `-t`, pour ne pas casser la redirection).

### 4.1 CREATE — Ajouter une nouvelle écoute

Insertion d'un nouveau document dans la collection `ecoutes`, représentant une écoute du morceau « Mr. Brightside » par `user_01`.

La méthode `insertOne()` ajoute un unique document à la collection. Le champ `_id` est fourni explicitement (`eco_99998`) plutôt que généré automatiquement par MongoDB, pour garder un identifiant lisible et cohérent avec la convention utilisée pour les autres écoutes.

**Requête :**

```
db.ecoutes.insertOne({
  _id: "eco_99998",
  id_utilisateur: "user_01",
  spotify_track_uri: "0eGsygTp906u18L0Oimnem",
  morceau: {
    titre: "Mr. Brightside",
    artiste: "The Killers",
    album: "Hot Fuss",
    genre: "indie"
  },
  plateforme: "android",
  ms_played: 222000,
  reason_start: "clickrow",
  reason_end: "trackdone",
  shuffle: false,
  skipped: false,
  date_ecoute: ISODate("2024-12-16T10:30:00Z")
});
```

### Résultat obtenu :

```
{ acknowledged: true, insertedId: 'eco_99998' }
```

```
streaming> db.ecoutes.insertOne({
  _id: "eco_99998",
  id_utilisateur: "user_01",
  spotify_track_uri: "0eGsygTp906u18L0Oimnem",
  morceau: {
    titre: "Mr. Brightside",
    artiste: "The Killers",
    album: "Hot Fuss",
    genre: "indie"
  },
  plateforme: "android",
  ms_played: 222000,
  reason_start: "clickrow",
  reason_end: "trackdone",
  shuffle: false,
  skipped: false,
  date_ecoute: ISODate("2024-12-16T10:30:00Z")
});
{ acknowledged: true, insertedId: 'eco_99998' }
streaming> █
```

## 4.2 READ — Rechercher les écoutes

Recherche des écoutes selon trois critères : artiste, genre, et période.

find() retourne les documents correspondant au filtre indiqué ; limit(5) restreint l'affichage aux cinq premiers résultats pour rester lisible. countDocuments() renvoie uniquement le nombre de documents correspondants, sans les charger en mémoire. Pour la période, les opérateurs \$gte et \$lt bornent l'intervalle de dates (du 1er janvier 2024 inclus au 1er janvier 2025 exclu).

### Requête :

```
// par artiste
db.ecoutes.find({ "morceau.artiste": "The Killers" }).limit(5);
db.ecoutes.countDocuments({ "morceau.artiste": "The Killers" });

// par genre
db.ecoutes.find({ "morceau.genre": "rock" }).limit(5);
db.ecoutes.countDocuments({ "morceau.genre": "rock" });

// par période (année 2024)
db.ecoutes.countDocuments({
  date_ecoute: { $gte: ISODate("2024-01-01"), $lt: ISODate("2025-01-01") }
});
```

## Résultat obtenu :

```
streaming> db.ecoutes.find({ "morceau.artiste": "The Killers" }).limit(5);
[
  {
    _id: 'eco_00001',
    id_utilisateur: 'user_01',
    spotify_track_uri: '6BnY0YochAURSXSr3d5N70',
    morceau: {
      titre: "Andy, You're A Star",
      artiste: 'The Killers',
      album: 'Hot Fuss',
      genre: 'indie'
    },
    plateforme: 'android',
    ms_played: 194080,
    reason_start: 'trackdone',
    reason_end: 'trackdone',
    shuffle: true,
    skipped: false,
    date_ecoute: ISODate('2023-10-05T03:08:51.000Z')
  },
  {
    _id: 'eco_00002',
    id_utilisateur: 'user_01',
    spotify_track_uri: '0qXWJhU46zurMYQoAnBxdG',
    morceau: {
      titre: 'Show You How',
      artiste: 'The Killers',
      album: 'Sawdust',
      genre: 'indie'
    },
    plateforme: 'android',
    ms_played: 166240,
    reason_start: 'trackdone',
    reason_end: 'trackdone',
    shuffle: true,
    skipped: false,
    date_ecoute: ISODate('2023-10-05T03:11:38.000Z')
  },
  {
    _id: 'eco_00003',
    id_utilisateur: 'user_01',
    spotify_track_uri: '2zdpz8sk3rkh0fLYabLjnmj',
    morceau: {
      titre: "I Can't Stay",
      artiste: 'The Killers',
      album: 'Day & Age - Bonus Tracks',
      genre: 'indie'
    },
  },
]
```

Par artiste :

```
[
  {
    plateforme: 'android',
    ms_played: 166240,
    reason_start: 'trackdone',
    reason_end: 'trackdone',
    shuffle: true,
    skipped: false,
    date_ecoute: ISODate('2023-10-05T03:11:38.000Z')
  },
  {
    _id: 'eco_00003',
    id_utilisateur: 'user_01',
    spotify_track_uri: '2zdpz8sk3rkh0fLYabLjmj',
    morceau: {
      titre: "I Can't Stay",
      artiste: 'The Killers',
      album: 'Day & Age - Bonus Tracks',
      genre: 'indie'
    },
    plateforme: 'android',
    ms_played: 185360,
    reason_start: 'trackdone',
    reason_end: 'trackdone',
    shuffle: true,
    skipped: false,
    date_ecoute: ISODate('2023-10-05T03:14:44.000Z')
  },
  {
    _id: 'eco_00004',
    id_utilisateur: 'user_01',
    spotify_track_uri: '1Tb4F7Ba2C8Yt3cMnYa3X3',
    morceau: {
      titre: 'Have All The Songs Been Written?',
      artiste: 'The Killers',
      album: 'Wonderful Wonderful',
      genre: 'indie'
    },
    plateforme: 'android',
    ms_played: 249946,
    reason_start: 'trackdone',
    reason_end: 'trackdone',
    shuffle: true,
    skipped: false,
    date_ecoute: ISODate('2023-10-05T03:18:54.000Z')
  },
  {
    _id: 'eco_00006',
    id_utilisateur: 'user_01',
    spotify_track_uri: '6PwjJ58I4t7Mae9xfZ9l9v',
    morceau: {
      titre: 'Somebody Told Me',
      artiste: 'The Killers',
      album: 'Hot Fuss',
      genre: 'indie'
    },
    plateforme: 'android',
    ms_played: 197200,
    reason_start: 'trackdone',
    reason_end: 'trackdone',
    shuffle: true,
    skipped: false,
    date_ecoute: ISODate('2023-10-05T03:32:48.000Z')
  }
]
```

Nombre d'écoute de l'artiste « The Killers » :

```
]
streaming> db.ecoutes.countDocuments({ "morceau.artiste": "The Killers" });
505
```

Par genre:

```
streaming> db.ecoutes.find({ "morceau.genre": "rock" }).limit(5);
[
  {
    _id: 'eco_00063',
    id_utilisateur: 'user_01',
    spotify_track_uri: '5HNCy40Ni5BZJFw1TKzRsC',
    morceau: {
      titre: 'Comfortably Numb',
      artiste: 'Pink Floyd',
      album: 'The Wall',
      genre: 'rock'
    },
    plateforme: 'android',
    ms_played: 194646,
    reason_start: 'trackdone',
    reason_end: 'logout',
    shuffle: true,
    skipped: false,
    date_ecoute: ISODate('2023-10-05T20:55:50.000Z')
  },
  {
    _id: 'eco_00103',
    id_utilisateur: 'user_01',
    spotify_track_uri: '4DZAH8eiqSAkMIE915D71U',
    morceau: {
      titre: 'Everybody's Trying To Be My Baby - Remastered 2009',
      artiste: 'The Beatles',
      album: 'Beatles For Sale - Remastered',
      genre: 'rock'
    },
    plateforme: 'android',
    ms_played: 4252,
    reason_start: 'playbtn',
    reason_end: 'fwdbtn',
    shuffle: true,
    skipped: true,
    date_ecoute: ISODate('2023-10-06T19:41:38.000Z')
  },
  {
    _id: 'eco_00104',
    id_utilisateur: 'user_01',
    spotify_track_uri: '6EHu0ufBeL6vk3TvVJB5qo',
    morceau: {
      titre: 'Please Please Me - Remastered 2009',
      artiste: 'The Beatles',
      album: 'Please Please Me',
      genre: 'rock'
    },
    plateforme: 'android',
    ms_played: 120853,
    reason_start: 'fwdbtn',
    reason_end: 'trackdone',
    shuffle: true,
    skipped: false,
    date_ecoute: ISODate('2023-10-06T19:43:38.000Z')
  },
]
```

```

{
  _id: 'eco_00105',
  id_utilisateur: 'user_01',
  spotify_track_uri: '1Dg4dFJr3HW7sbA7vPejre',
  morceau: {
    titre: 'Eight Days A Week - Remastered 2009',
    artiste: 'The Beatles',
    album: 'Beatles For Sale - Remastered',
    genre: 'rock'
  },
  plateforme: 'android',
  ms_played: 163600,
  reason_start: 'trackdone',
  reason_end: 'trackdone',
  shuffle: true,
  skipped: false,
  date_ecoute: ISODate('2023-10-06T19:46:23.000Z')
},
{
  _id: 'eco_00106',
  id_utilisateur: 'user_01',
  spotify_track_uri: '3BQHpfGAp4l80e1XslIjNI',
  morceau: {
    titre: 'Yesterday - Remastered 2009',
    artiste: 'The Beatles',
    album: 'Help!',
    genre: 'rock'
  },
  plateforme: 'android',
  ms_played: 125666,
  reason_start: 'trackdone',
  reason_end: 'trackdone',
  shuffle: true,
  skipped: false,
  date_ecoute: ISODate('2023-10-06T19:48:29.000Z')
}
]
streaming>

```

Nombre d'écoutes avec genre rock :

```

streaming> db.ecoutes.countDocuments({ "morceau.genre": "rock" });
3416

```

Par période :

```

streaming> db.ecoutes.countDocuments({
|   date_ecoute: {
|     $gte: ISODate("2024-01-01"),
|     $lt: ISODate("2025-01-01")
|   }
| });
9122

```

### 4.3 UPDATE — Modifier les métadonnées d'un morceau

Correction du genre d'un morceau identifié par son `spotify_track_uri`. Le morceau étant imbriqué dans chaque écoute, ses métadonnées sont dupliquées dans tous les documents concernés : la correction doit donc utiliser `updateMany` et non `updateOne` pour rester cohérente sur l'ensemble des occurrences.

L'opérateur `$set` ne modifie que le champ `morceau.genre` : le reste du document (titre, artiste, album, métadonnées de lecture) n'est pas touché. `updateMany()` applique cette modification à tous les documents correspondant au filtre, contrairement à `updateOne()` qui se limiterait au premier trouvé.

**Requête :**

```
db.ecoutes.updateMany(
  { spotify_track_uri: "5BckPAYcKEJuYs1eV1BHHe" },
  { $set: { "morceau.genre": "alternative" } }
);
```

**Résultat obtenu :**

```
streaming> db.ecoutes.updateMany(
|   { spotify_track_uri: "5BckPAYcKEJuYs1eV1BHHe" },
|   { $set: { "morceau.genre": "alternative" } }
| );
|
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 25,
|   modifiedCount: 25,
|   upsertedCount: 0
| }
streaming>
```

*matchedCount reflète les 25 documents partageant ce `spotify_track_uri` ; modifiedCount = 25 signifie que ces documents sont tous modifiés.*

### 4.4 DELETE — Supprimer les écoutes d'un utilisateur

Suppression de toutes les écoutes de `user_16`, puis suppression explicite de son profil. MongoDB ne réalise aucune suppression en cascade (contrairement à un `ON DELETE CASCADE` en SQL) : l'intégrité référentielle entre `ecoutes` et `utilisateurs` est donc à la charge de l'application, d'où les deux requêtes distinctes.

`deleteMany()` supprime tous les documents correspondant au filtre indiqué (ici, toutes les écoutes dont `id_utilisateur` vaut `user_16`), tandis que `deleteOne()` ne supprime qu'un seul document, ici ciblé directement par son `_id`.

**Requête :**

```
db.ecoutes.deleteMany({ id_utilisateur: "user_16" });
db.utilisateurs.deleteOne({ _id: "user_16" });
```

Résultat obtenu :

```
deleteMany
streaming> db.ecoutes.deleteMany({ id_utilisateur: "user_16" });
{ acknowledged: true, deletedCount: 750 }
```

```
deleteOne
streaming> db.utilisateurs.deleteOne({ _id: "user_16" });
{ acknowledged: true, deletedCount: 1 }
```

*Les données ont été restaurées immédiatement après ce test via python scripts/import\_mongodb.py (script idempotent) afin de retrouver les 12 000 écoutes et 16 utilisateurs pour la suite du projet.*

## 5. Analyses

Les analyses ont été implémentées dans le fichier queries/analyses.js sous forme de pipelines d'agrégation (aggregate), exécutés dans mongosh en mode interactif pour permettre les captures d'écran, requête par requête.

### 5.1 Analyses minimales attendues

#### 5.1.1 Morceaux les plus écoutés

Regroupement des écoutes par morceau (titre + artiste), triées par nombre d'écoutes décroissant, limité au top 10.

\$group régule les documents par morceau (combinaison titre + artiste) et \$sum: 1 compte le nombre d'écoutes de chaque groupe. \$sort trie ensuite par nombre d'écoutes décroissant, et \$limit ne conserve que les 10 premiers résultats.

**Requête :**

```
db.ecoutes.aggregate([
  { $group: {
    _id: { titre: "$morceau.titre", artiste: "$morceau.artiste" },
    nb_ecoutes: { $sum: 1 }
  } },
  { $sort: { nb_ecoutes: -1 } },
  { $limit: 10 }
]);
```

Résultat obtenu :

```
streaming> db.ecoutes.aggregate([
  { $group: {
    _id: { titre: "$morceau.titre", artiste: "$morceau.artiste" },
    nb_ecoutes: { $sum: 1 }
  } },
  { $sort: { nb_ecoutes: -1 } },
  { $limit: 10 }
]);

{
  _id: { titre: 'You Sexy Thing', artiste: 'Hot Chocolate' },
  nb_ecoutes: 51
},
{
  _id: { titre: 'Why Did It Have To Be Me?', artiste: 'ABBA' },
  nb_ecoutes: 39
},
{
  _id: {
    titre: 'The Return of the King (feat. Sir James Galway, Viggo Mortensen and Renee Fleming)',
    artiste: 'Howard Shore'
  },
  nb_ecoutes: 29
},
{
  _id: { titre: 'Superdeli', artiste: 'Caloncho' }, nb_ecoutes: 28 },
{
  _id: { titre: 'Qué Bonito Es Querer', artiste: 'Manuel Carrasco' },
  nb_ecoutes: 28
},
{
  _id: { titre: 'Does Your Mother Know', artiste: 'ABBA' },
  nb_ecoutes: 27
},
{
  _id: { titre: 'Knowing Me, Knowing You', artiste: 'ABBA' },
  nb_ecoutes: 27
},
{
  _id: { titre: 'Dancing Queen', artiste: 'ABBA' }, nb_ecoutes: 27 },
{
  _id: { titre: 'Waterloo', artiste: 'ABBA' }, nb_ecoutes: 26 },
{
  _id: { titre: 'Take A Chance On Me', artiste: 'ABBA' },
  nb_ecoutes: 25
}
}
```

### 5.1.2 Temps total d'écoute par genre

Somme du temps écouté (ms\_played) regroupée par genre musical.

\$group régule ici les documents par genre. \$sum: "\$ms\_played" additionne le temps réellement écouté (en millisecondes) pour chaque genre, au lieu de se contenter de compter les occurrences comme dans les analyses précédentes.

**Requête :**

```
db.ecoutes.aggregate([
  { $group: {
    _id: "$morceau.genre",
    temps_total_ms: { $sum: "$ms_played" }
  } },
  { $sort: { temps_total_ms: -1 } }
]);
```

Résultat obtenu :

```
streaming> db.ecoutes.aggregate([
  { $group: {
    _id: "$morceau.genre",
    temps_total_ms: { $sum: "$ms_played" }
  } },
  { $sort: { temps_total_ms: -1 } }
]);
[
  { _id: 'rock', temps_total_ms: 559395451 },
  { _id: 'singer-songwriter', temps_total_ms: 231483446 },
  { _id: 'indie', temps_total_ms: 219317859 },
  { _id: 'classical', temps_total_ms: 179650406 },
  { _id: 'pop', temps_total_ms: 174993115 },
  { _id: 'folk', temps_total_ms: 138352410 },
  { _id: 'hip-hop', temps_total_ms: 87189350 },
  { _id: 'Unknown', temps_total_ms: 72246057 },
  { _id: 'latin', temps_total_ms: 70956323 },
  { _id: 'blues', temps_total_ms: 52970084 },
  { _id: 'funk', temps_total_ms: 52178140 },
  { _id: 'jazz', temps_total_ms: 48301759 },
  { _id: 'country', temps_total_ms: 43296661 },
  { _id: 'electronic', temps_total_ms: 43032036 },
  { _id: 'soul', temps_total_ms: 40210729 },
  { _id: 'disco', temps_total_ms: 28165016 },
  { _id: 'psychedelic', temps_total_ms: 15625198 },
  { _id: 'rap', temps_total_ms: 14491644 },
  { _id: 'house', temps_total_ms: 13223560 },
  { _id: 'alternative', temps_total_ms: 12590668 }
]
Type "it" for more
```

### 5.1.3 Artistes les plus écoutés

Regroupement des écoutes par artiste, triées par nombre d'écoutes décroissant, limité au top 10.

Même logique que pour les morceaux : \$group règle par artiste, \$sum: 1 compte les écoutes de chacun, puis \$sort et \$limit ne conservent que le top 10.

**Requête :**

```
db.ecoutes.aggregate([
  { $group: {
    _id: "$morceau.artiste",
    nb_ecoutes: { $sum: 1 }
  } },
  { $sort: { nb_ecoutes: -1 } },
  { $limit: 10 }
]);
```

Résultat obtenu :

```
streaming> db.ecoutes.aggregate([
  { $group: {
    _id: "$morceau.artiste",
    nb_ecoutes: { $sum: 1 }
  } },
  { $sort: { nb_ecoutes: -1 } },
  { $limit: 10 }
]);

[
  { _id: 'The Beatles', nb_ecoutes: 675 },
  { _id: 'The Killers', nb_ecoutes: 524 },
  { _id: 'John Mayer', nb_ecoutes: 477 },
  { _id: 'ABBA', nb_ecoutes: 318 },
  { _id: 'Howard Shore', nb_ecoutes: 223 },
  { _id: 'Bob Dylan', nb_ecoutes: 196 },
  { _id: 'Paul McCartney', nb_ecoutes: 195 },
  { _id: 'Joaquín Sabina', nb_ecoutes: 164 },
  { _id: 'Billy Joel', nb_ecoutes: 156 },
  { _id: 'Caloncho', nb_ecoutes: 139 }
]
```

#### 5.1.4 Répartition des écoutes par plateforme

Nombre d'écoutes regroupé par plateforme (android, ios, web, ...).

\$group régle les écoutes par plateforme et \$sum: 1 compte le nombre d'écoutes de chacune ; \$sort classe les plateformes par nombre d'écoutes décroissant.

**Requête :**

```
db.ecoutes.aggregate([
  { $group: {
    _id: "$plateforme",
    nb_ecoutes: { $sum: 1 }
  } },
  { $sort: { nb_ecoutes: -1 } }
]);
```

Résultat obtenu :

```
streaming> db.ecoutes.aggregate([
  { $group: {
    _id: "$plateforme",
    nb_ecoutes: { $sum: 1 }
  } },
  { $sort: { nb_ecoutes: -1 } }
]);

[
  { _id: 'android', nb_ecoutes: 11610 },
  { _id: 'mac', nb_ecoutes: 390 }
]
```

L'échantillon ne contient que deux plateformes, Android représentant 96,8 % des écoutes. Cet écart est expliqué au §7.4.

### 5.1.5 Évolution des écoutes dans le temps

Nombre d'écoutes regroupé par année et par mois, trié chronologiquement.

Les opérateurs \$year et \$month extraient respectivement l'année et le mois du champ date\_ecoute, formant ainsi une clé de regroupement composite (année, mois). \$sort ordonne ensuite les résultats chronologiquement.

**Requête :**

```
db.ecoutes.aggregate([
  { $group: {
    _id: { annee: { $year: "$date_ecoute" }, mois: { $month: "$date_ecoute" } },
    nb_ecoutes: { $sum: 1 }
  } },
  { $sort: { "_id.annee": 1, "_id.mois": 1 } }
]);
```

**Résultat obtenu :**

```
streaming> db.ecoutes.aggregate([
  { $group: {
    _id: { annee: { $year: "$date_ecoute" }, mois: { $month: "$date_ecoute" } },
    nb_ecoutes: { $sum: 1 }
  } },
  { $sort: { "_id.annee": 1, "_id.mois": 1 } }
]);

[
  { _id: { annee: 2023, mois: 10 }, nb_ecoutes: 570 },
  { _id: { annee: 2023, mois: 11 }, nb_ecoutes: 785 },
  { _id: { annee: 2023, mois: 12 }, nb_ecoutes: 775 },
  { _id: { annee: 2024, mois: 1 }, nb_ecoutes: 1449 },
  { _id: { annee: 2024, mois: 2 }, nb_ecoutes: 924 },
  { _id: { annee: 2024, mois: 3 }, nb_ecoutes: 700 },
  { _id: { annee: 2024, mois: 4 }, nb_ecoutes: 619 },
  { _id: { annee: 2024, mois: 5 }, nb_ecoutes: 569 },
  { _id: { annee: 2024, mois: 6 }, nb_ecoutes: 524 },
  { _id: { annee: 2024, mois: 7 }, nb_ecoutes: 318 },
  { _id: { annee: 2024, mois: 8 }, nb_ecoutes: 932 },
  { _id: { annee: 2024, mois: 9 }, nb_ecoutes: 962 },
  { _id: { annee: 2024, mois: 10 }, nb_ecoutes: 1300 },
  { _id: { annee: 2024, mois: 11 }, nb_ecoutes: 1425 },
  { _id: { annee: 2024, mois: 12 }, nb_ecoutes: 148 }
]
```

## 5.2 Analyses complémentaires (au moins 3 exigées)

### 5.2.1 Genre préféré de chaque utilisateur

Pour chaque utilisateur, identification du genre musical le plus écouté (double \$group : par utilisateur+genre, puis par utilisateur en gardant le genre en tête après tri).

Le premier \$group compte les écoutes par couple (utilisateur, genre). Le \$sort intermédiaire place le genre le plus écouté en tête pour chaque utilisateur ; le second \$group ne conserve alors que la première valeur rencontrée grâce à \$first, qui correspond donc au genre dominant de chaque utilisateur.

**Requête :**

```

db.ecoutes.aggregate([
  { $group: {
    _id: { utilisateur: "$id_utilisateur", genre: "$morceau.genre" },
    nb: { $sum: 1 }
  } },
  { $sort: { "_id.utilisateur": 1, nb: -1 } },
  { $group: {
    _id: "$_id.utilisateur",
    genre_prefere: { $first: "$_id.genre" },
    nb_ecoutes: { $first: "$nb" }
  } },
  { $sort: { _id: 1 } }
]);

```

Résultat obtenu :

```

streaming> db.ecoutes.aggregate([
  { $group: {
    _id: { utilisateur: "$id_utilisateur", genre: "$morceau.genre" },
    nb: { $sum: 1 }
  } },
  { $sort: { "_id.utilisateur": 1, nb: -1 } },
  { $group: {
    _id: "$_id.utilisateur",
    genre_prefere: { $first: "$_id.genre" },
    nb_ecoutes: { $first: "$nb" }
  } },
  { $sort: { _id: 1 } }
]);

[
  { _id: 'user_01', genre_prefere: 'rock', nb_ecoutes: 190 },
  { _id: 'user_02', genre_prefere: 'rock', nb_ecoutes: 234 },
  { _id: 'user_03', genre_prefere: 'rock', nb_ecoutes: 201 },
  { _id: 'user_04', genre_prefere: 'rock', nb_ecoutes: 117 },
  { _id: 'user_05', genre_prefere: 'rock', nb_ecoutes: 212 },
  { _id: 'user_06', genre_prefere: 'indie', nb_ecoutes: 188 },
  { _id: 'user_07', genre_prefere: 'singer-songwriter', nb_ecoutes: 204 },
  { _id: 'user_08', genre_prefere: 'rock', nb_ecoutes: 239 },
  { _id: 'user_09', genre_prefere: 'rock', nb_ecoutes: 263 },
  { _id: 'user_10', genre_prefere: 'rock', nb_ecoutes: 260 },
  { _id: 'user_11', genre_prefere: 'rock', nb_ecoutes: 220 },
  { _id: 'user_12', genre_prefere: 'rock', nb_ecoutes: 240 },
  { _id: 'user_13', genre_prefere: 'rock', nb_ecoutes: 284 },
  { _id: 'user_14', genre_prefere: 'hip-hop', nb_ecoutes: 298 },
  { _id: 'user_15', genre_prefere: 'rock', nb_ecoutes: 255 },
  { _id: 'user_16', genre_prefere: 'rock', nb_ecoutes: 236 }
]

```

### 5.2.2 Heures de forte écoute

Regroupement des écoutes par heure de la journée (0-23h) pour identifier les créneaux de forte activité.

\$hour extrait l'heure de la date d'écoute (0 à 23). \$divide convertit le temps cumulé de millisecondes en minutes, puis \$project restructure le résultat et arrondit cette durée avec \$round pour l'affichage.

**Requête :**

```

db.ecoutes.aggregate([
  { $group: {

```

```

    _id: { $hour: "$date_ecoute" },
    nb_ecoutes: { $sum: 1 },
    duree_totale_min: { $sum: { $divide: ["$ms_played", 60000] } }
  } },
  { $project: {
    _id: 0,
    heure: "$_id",
    nb_ecoutes: 1,
    duree_totale_min: { $round: ["$duree_totale_min", 0] }
  } },
  { $sort: { nb_ecoutes: -1 } }
]).toArray();

```

Résultat obtenu :

```

streaming> db.ecoutes.aggregate([
  { $group: {
    _id: { $hour: "$date_ecoute" },
    nb_ecoutes: { $sum: 1 },
    duree_totale_min: { $sum: { $divide: ["$ms_played", 60000] } }
  } },
  { $project: {
    _id: 0,
    heure: "$_id",
    nb_ecoutes: 1,
    duree_totale_min: { $round: ["$duree_totale_min", 0] }
  } },
  { $sort: { nb_ecoutes: -1 } }
]).toArray();
[
  { nb_ecoutes: 945, heure: 0, duree_totale_min: 2453 },
  { nb_ecoutes: 911, heure: 19, duree_totale_min: 2875 },
  { nb_ecoutes: 891, heure: 18, duree_totale_min: 2924 },
  { nb_ecoutes: 891, heure: 23, duree_totale_min: 2310 },
  { nb_ecoutes: 890, heure: 22, duree_totale_min: 2258 },
  { nb_ecoutes: 875, heure: 20, duree_totale_min: 2558 },
  { nb_ecoutes: 866, heure: 2, duree_totale_min: 2551 },
  { nb_ecoutes: 816, heure: 1, duree_totale_min: 2415 },
  { nb_ecoutes: 778, heure: 21, duree_totale_min: 2124 },
  { nb_ecoutes: 728, heure: 17, duree_totale_min: 2286 },
  { nb_ecoutes: 636, heure: 3, duree_totale_min: 1836 },
  { nb_ecoutes: 584, heure: 5, duree_totale_min: 1856 },
  { nb_ecoutes: 554, heure: 4, duree_totale_min: 1740 },
  { nb_ecoutes: 501, heure: 6, duree_totale_min: 1679 },
  { nb_ecoutes: 348, heure: 16, duree_totale_min: 1111 },
  { nb_ecoutes: 219, heure: 7, duree_totale_min: 744 },
  { nb_ecoutes: 127, heure: 15, duree_totale_min: 409 },
  { nb_ecoutes: 105, heure: 9, duree_totale_min: 289 },
  { nb_ecoutes: 96, heure: 8, duree_totale_min: 346 },
  { nb_ecoutes: 83, heure: 14, duree_totale_min: 287 },
  { nb_ecoutes: 75, heure: 10, duree_totale_min: 179 },
  { nb_ecoutes: 47, heure: 13, duree_totale_min: 186 },
  { nb_ecoutes: 19, heure: 12, duree_totale_min: 66 },
  { nb_ecoutes: 16, heure: 11, duree_totale_min: 58 }
]
streaming>

```

### 5.2.3 Taux de réécoute des morceaux populaires

Pour les morceaux les plus écoutés, calcul du ratio nombre d'écoutes / nombre d'utilisateurs distincts ayant écouté ce morceau. Un ratio élevé indique que le morceau est réécouté plusieurs fois par les mêmes utilisateurs plutôt que découvert une seule fois par beaucoup d'utilisateurs différents.

\$addToSet collecte, pour chaque morceau, les identifiants distincts des utilisateurs l'ayant écouté (sans doublon). \$size compte ensuite ce nombre d'utilisateurs uniques, utilisé au dénominateur du taux de réécoute calculé par \$divide.

**Requête :**

```
db.ecoutes.aggregate([
  { $group: {
    _id: { titre: "$morceau.titre", artiste: "$morceau.artiste" },
    nb_ecoutes: { $sum: 1 },
    utilisateurs_distincts: { $addToSet: "$id_utilisateur" }
  } },
  { $project: {
    nb_ecoutes: 1,
    nb_utilisateurs: { $size: "$utilisateurs_distincts" },
    taux_reecoute: { $divide: ["$nb_ecoutes", { $size: "$utilisateurs_distincts" }] }
  } },
  { $sort: { nb_ecoutes: -1 } },
  { $limit: 10 }
]);
```

## Résultat obtenu :

```
streaming> db.ecoutes.aggregate([
  { $group: {
    _id: { titre: "$morceau.titre", artiste: "$morceau.artiste" },
    nb_ecoutes: { $sum: 1 },
    utilisateurs_distincts: { $addToSet: "$id_utilisateur" }
  } },
  { $project: {
    nb_ecoutes: 1,
    nb_utilisateurs: { $size: "$utilisateurs_distincts" },
    taux_reecoute: { $divide: [ "$nb_ecoutes", { $size: "$utilisateurs_distincts" } ] }
  } },
  { $sort: { nb_ecoutes: -1 } },
  { $limit: 10 }
]);

[
  {
    _id: { titre: 'You Sexy Thing', artiste: 'Hot Chocolate' },
    nb_ecoutes: 51,
    nb_utilisateurs: 10,
    taux_reecoute: 5.1
  },
  {
    _id: { titre: 'Why Did It Have To Be Me?', artiste: 'ABBA' },
    nb_ecoutes: 39,
    nb_utilisateurs: 7,
    taux_reecoute: 5.571428571428571
  },
  {
    _id: {
      titre: 'The Return of the King (feat. Sir James Galway, Viggo Mortensen and Renee Fleming)',
      artiste: 'Howard Shore'
    },
    nb_ecoutes: 29,
    nb_utilisateurs: 10,
    taux_reecoute: 2.9
  },
  {
    _id: { titre: 'Superdeli', artiste: 'Caloncho' },
    nb_ecoutes: 28,
    nb_utilisateurs: 7,
    taux_reecoute: 4
  },
  {
    _id: { titre: 'Qué Bonito Es Querer', artiste: 'Manuel Carrasco' },
    nb_ecoutes: 28,
    nb_utilisateurs: 4,
    taux_reecoute: 7
  },
  {
    _id: { titre: 'Dancing Queen', artiste: 'ABBA' },
    nb_ecoutes: 27,
    nb_utilisateurs: 10,
    taux_reecoute: 2.7
  },
  {
    _id: { titre: 'Knowing Me, Knowing You', artiste: 'ABBA' },
    nb_ecoutes: 27,
    nb_utilisateurs: 8,
    taux_reecoute: 3.375
  },
  {
    _id: { titre: 'Does Your Mother Know', artiste: 'ABBA' },
    nb_ecoutes: 27,
    nb_utilisateurs: 8,
    taux_reecoute: 3.375
  },
  {
    _id: { titre: 'Waterloo', artiste: 'ABBA' },
    nb_ecoutes: 26,
    nb_utilisateurs: 9,
    taux_reecoute: 2.888888888888889
  },
  {
    _id: { titre: 'boy', artiste: 'The Killers' },
    nb_ecoutes: 25,
    nb_utilisateurs: 11,
    taux_reecoute: 2.272727272727273
  }
]
```

## 6. Indexation

Les requêtes d'indexation ont été regroupées dans le fichier queries/indexation.js, exécuté dans mongosh.

La stratégie est illustrée sur le champ morceau.artiste, déjà utilisé dans les requêtes de recherche du §4.2. La comparaison avant/après repose sur explain("executionStats") appliqué à la même requête, afin d'observer le passage d'un scan complet de la collection (COLLSCAN) à un scan d'index ciblé (IXSCAN).

## 6.1 Avant indexation — plan d'exécution

Analyse du plan d'exécution de la requête de recherche par artiste, sans index dédié sur ce champ (en dehors de l'index par défaut sur `_id`).

`explain("executionStats")` exécute la requête et retourne, en plus des résultats attendus, des statistiques détaillées sur le plan choisi par MongoDB : type de scan utilisé, nombre de documents et de clés d'index examinés, et temps d'exécution.

**Requête :**

```
db.ecoutes.find({ "morceau.artiste": "The Killers" }).explain("executionStats");
```

**Résultat obtenu :**

```
streaming> db.ecoutes.find({ "morceau.artiste": "The Killers" }).explain("executionStats");
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'streaming.ecoutes',
    indexFilterSet: false,
    parsedQuery: { 'morceau.artiste': { '$eq': 'The Killers' } },
    queryHash: '5ABA1154',
    planCacheKey: '5ABA1154',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    winningPlan: {
      stage: 'COLLSCAN',
      filter: { 'morceau.artiste': { '$eq': 'The Killers' } },
      direction: 'forward'
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 524,
    executionTimeMillis: 17,
    totalKeysExamined: 0,
    totalDocsExamined: 12000,
    executionStages: {
      stage: 'COLLSCAN',
      filter: { 'morceau.artiste': { '$eq': 'The Killers' } },
      nReturned: 524,
      executionTimeMillisEstimate: 5,
      works: 12001,
      advanced: 524,
      needTime: 11476,
      needYield: 0,
      saveState: 12,
      restoreState: 12,
      isEOF: 1,
      direction: 'forward',
      docsExamined: 12000
    }
  },
  command: {
    find: 'ecoutes',
    filter: { 'morceau.artiste': 'The Killers' },
    '$db': 'streaming'
  },
  serverInfo: {
    host: '1f12f76b80df',
    port: 27017,
    version: '7.0.37',
    gitVersion: '9d30419d900008ba3ecf2a14546b236f2158b65b'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFrameworkControl: 'forceClassicEngine'
  },
  ok: 1
}
```

On remarque que `totalDocsExamined` vaut 12 000, soit la totalité des documents de la collection, MongoDB a donc examiné chaque document un par un pour ne retenir que les écoutes de « The Killers ».

## 6.2 Création des index

Création d'un index simple ascendant sur `id_utilisateur`, `date_ecoute`, `genre` et `artiste`.

| Champ                        | Cardinalité | Intérêt de l'index                                                                                                                                            |
|------------------------------|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>morceau.artiste</code> | 1 113       | Très élevée. Filtrer sur un artiste élimine presque toute la collection. C'est le meilleur candidat, retenu pour la démonstration avant/après.                |
| <code>date_ecoute</code>     | ~12 000     | Quasi unique. Indispensable pour les requêtes par période et pour les tris chronologiques.                                                                    |
| <code>id_utilisateur</code>  | 16          | Faible, mais chaque utilisateur ne représente que 750 écoutes (6 % de la collection). L'index reste utile, notamment pour le <code>deleteMany</code> du §4.4. |
| <code>morceau.genre</code>   | 25          | Faible et déséquilibrée : quelques genres concentrent la majorité des écoutes. Le gain est donc limité pour les genres dominants, réel pour les genres rares. |

`createIndex({ champ: 1 })` crée un index simple sur le champ indiqué, trié par ordre croissant (la valeur -1 donnerait un ordre décroissant). Chacun des quatre index est créé indépendamment, sur un seul champ à la fois.

**Requête :**

```
db.ecoutes.createIndex({ "morceau.artiste": 1 });
db.ecoutes.createIndex({ "date_ecoute": 1 });
db.ecoutes.createIndex({ "id_utilisateur": 1 });
db.ecoutes.createIndex({ "morceau.genre": 1 });
```

**Résultat obtenu :**

```
streaming> db.ecoutes.createIndex({ "morceau.artiste": 1 });
morceau.artiste_1
streaming> db.ecoutes.createIndex({ "date_ecoute": 1 });
date_ecoute_1
streaming> db.ecoutes.createIndex({ "id_utilisateur": 1 });
id_utilisateur_1
streaming> db.ecoutes.createIndex({ "morceau.genre": 1 });
morceau.genre_1
```

`getIndexes()` liste l'ensemble des index existants sur la collection, y compris l'index par défaut sur `_id` créé automatiquement par MongoDB.

**Affiche Tous les index :**

```
db.ecoutes.getIndexes();
```

Résultat obtenu :

```
streaming> db.ecoutes.getIndexes();
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { 'morceau.artiste': 1 }, name: 'morceau.artiste_1' },
  { v: 2, key: { date_ecoute: 1 }, name: 'date_ecoute_1' },
  { v: 2, key: { id_utilisateur: 1 }, name: 'id_utilisateur_1' },
  { v: 2, key: { 'morceau.genre': 1 }, name: 'morceau.genre_1' }
]
```

### 6.3 Comparaison après indexation (index morceau.artiste)

Même requête, ré-exécutée après création de l'index, pour comparaison directe.

Même requête explain("executionStats") qu'au §6.1, ré-exécutée après la création de l'index sur morceau.artiste, pour une comparaison directe du plan d'exécution.

Requête :

```
db.ecoutes.find({ "morceau.artiste": "The Killers" }).explain("executionStats");
```

Résultat obtenu :

```
streaming> db.ecoutes.find({ "morceau.artiste": "The Killers" }).explain("executionStats");
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'streaming.ecoutes',
    indexFilterSet: false,
    parsedQuery: { 'morceau.artiste': { '$eq': 'The Killers' } },
    queryHash: '5ABA1154',
    planCacheKey: '2745CA33',
    optimizationTimeMillis: 5,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    winningPlan: {
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { 'morceau.artiste': 1 },
        indexName: 'morceau.artiste_1',
        isMultiKey: false,
        multiKeyPaths: { 'morceau.artiste': [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { 'morceau.artiste': [ ['The Killers', 'The Killers'] ] }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 524,
    executionTimeMillis: 9,
    totalKeysExamined: 524,
    totalDocsExamined: 524,
    executionStages: {
      stage: 'FETCH',
      nReturned: 524,
      executionTimeMillisEstimate: 10,
      works: 525,
      advanced: 524,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      docsExamined: 524,
      alreadyHasObj: 0,

```

```

inputStage: {
  stage: 'IXSCAN',
  nReturned: 524,
  executionTimeMillisEstimate: 0,
  works: 525,
  advanced: 524,
  needTime: 0,
  needYield: 0,
  saveState: 0,
  restoreState: 0,
  isEOF: 1,
  keyPattern: { 'morceau.artiste': 1 },
  indexName: 'morceau.artiste_1',
  isMultiKey: false,
  multiKeyPaths: { 'morceau.artiste': [] },
  isUnique: false,
  isSparse: false,
  isPartial: false,
  indexVersion: 2,
  direction: 'forward',
  indexBounds: { 'morceau.artiste': [ '['The Killers', 'The Killers']' ] },
  keysExamined: 524,
  seeks: 1,
  dupsTested: 0,
  dupsDropped: 0
}
},
command: {
  find: 'ecoutes',
  filter: { 'morceau.artiste': 'The Killers' },
  '$db': 'streaming'
},
serverInfo: {
  host: '1f12f76b80df',
  port: 27017,
  version: '7.0.37',
  gitVersion: '9d30419d900008ba3ecf2a14546b236f2158b65b'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFrameworkControl: 'forceClassicEngine'
},
ok: 1
}

```

On remarque que totalDocsExamined vaut maintenant 524, c'est-à-dire le même nombre que nReturned. MongoDB ouvre donc uniquement les documents dont il a besoin, sans en examiner d'autres inutilement. Le COLLSCAN a été remplacé par un IXSCAN, suivi d'un FETCH ciblé.

## 6.4 Comparaison avant / après

| Indicateur          | Avant(COLLSCAN) | Après(IXSCAN)         |
|---------------------|-----------------|-----------------------|
| stage               | <b>COLLSCAN</b> | <b>FETCH + IXSCAN</b> |
| totalDocsExamined   | <b>12 000</b>   | <b>524</b>            |
| totalKeysExamined   | <b>0</b>        | <b>524</b>            |
| nReturned           | <b>524</b>      | <b>524</b>            |
| executionTimeMillis | <b>17 ms</b>    | <b>9 ms</b>           |

## Interprétation :

Avant l'indexation, MongoDB doit lire les 12 000 documents de la collection pour n'en garder que 524, ceux de « The Killers ». Il ouvre donc environ 23 documents pour en retourner un seul utile.

Après la création de l'index sur morceau.artiste, le moteur utilise un IXSCAN puis un FETCH ciblé. Il n'examine plus que les 524 documents concernés : totalDocsExamined, totalKeysExamined et nReturned valent tous 524. Aucun document inutile n'est lu.

Ce gain a un prix. L'index occupe de la place sur le disque et en mémoire. Surtout, il doit être mis à jour à chaque écriture (insert, update, delete) dans la collection ecoutes, ce qui ralentit un peu ces opérations. C'est le compromis habituel de l'indexation : on gagne en lecture, on perd en écriture.

Dans notre cas, ce compromis est intéressant : sur une plateforme de streaming, on lit les données bien plus souvent qu'on ne les écrit.

## 7. Résultats obtenus

### 7.1 Import et modèle

12 000 écoutes et 16 utilisateurs importés, conformes au volume conseillé de 8 000 à 15 000. Deux collections ont été créées : ecoutes, avec le morceau imbriqué, et utilisateurs, référencée par id\_utilisateur.

L'enrichissement par Last.fm couvre 96,7 % des écoutes. 25 genres apparaissent en base, soit les 24 genres de la liste blanche plus la valeur Unknown, qui concerne 392 écoutes.

### 7.2 Opérations CRUD

Les quatre opérations ont été validées dans mongosh. Deux résultats méritent d'être relevés.

Le updateMany a modifié 25 documents pour un seul morceau. C'est la conséquence directe de l'imbrication : les métadonnées sont dupliquées dans chaque écoute.

Le deleteMany a supprimé les 750 écoutes de user\_16 sans toucher à son profil. MongoDB ne réalisant aucune suppression en cascade, une seconde requête a été nécessaire.

### 7.3 Analyses

| Indicateur             | Résultat                       | Valeur          |
|------------------------|--------------------------------|-----------------|
| Morceau le plus écouté | You Sexy Thing — Hot Chocolate | 51 écoutes      |
| Artiste le plus écouté | The Beatles                    | 675 écoutes     |
| Genre dominant (temps) | rock                           | 155 h sur 592 h |
| Temps d'écoute total   | 12 000 écoutes                 | 592 heures      |
| Mois le plus actif     | janvier 2024                   | 1 449 écoutes   |
| Heures de pic          | 00 h, 19 h, 18 h               | 945 / 911 / 891 |

Répartition du temps d'écoute par genre (top 3).

| Genre             | Temps total | Écoutes |
|-------------------|-------------|---------|
| rock              | 155,4 h     | 3 416   |
| singer-songwriter | 64,3 h      | 1 171   |
| indie             | 60,9 h      | 1 262   |

Le rock domine largement, avec 26 % du temps total d'écoute et 28 % des écoutes. Il est également le genre préféré de 13 utilisateurs sur 16. Les trois autres se répartissent entre indie, hip-hop et singer-songwriter.

Cette concentration est un artefact de la simulation : les utilisateurs étant des tranches chronologiques d'un même historique, ils partagent nécessairement les mêmes goûts dominants.

#### 7.4 Répartition par plateforme

| Plateforme | Écoutes | Part   |
|------------|---------|--------|
| android    | 11 610  | 96,8 % |
| mac        | 390     | 3,2 %  |

Le jeu de données brut comptait six plateformes. L'échantillon n'en contient plus que deux, et Android en représente 96,8 %.

Ce n'est pas une anomalie mais une conséquence de l'échantillonnage chronologique : les plateformes web player, windows et iOS n'apparaissent que dans les années antérieures à 2023, exclues de l'échantillon. L'analyse par plateforme exigée par l'énoncé produit donc un résultat peu discriminant sur ce périmètre.

#### 7.5 Indexation

L'index sur `morceau.artiste` fait passer le nombre de documents examinés de 12 000 à 524, soit exactement le nombre de documents retournés (§6.4).

#### 7.6 Un résultat à interpréter avec prudence

L'heure de pic la plus élevée est minuit (945 écoutes), devant 19 h et 18 h. Ce pic à 00 h est probablement lié au fuseau horaire : les dates sont stockées en UTC, alors que les écoutes ont eu lieu en heure locale. Les créneaux de 18 h et 19 h, eux, correspondent à un usage attendu en fin de journée.

## 8. Tableau de bord (livrable complémentaire)

Un tableau de bord web a été développé en complément des requêtes exécutées dans mongosh. Ce n'est pas un livrable exigé : l'énoncé cite l'interface comme un fichier complémentaire éventuel. Son but est d'afficher les analyses du §5 sous forme de graphiques, plus lisibles que les sorties du shell.

### 8.1 Stack technique

| Composant                 | Rôle                                                        |
|---------------------------|-------------------------------------------------------------|
| Streamlit                 | Framework d'application web en Python                       |
| PyMongo                   | Connexion à MongoDB et exécution des pipelines d'agrégation |
| Altair                    | Rendu des graphiques                                        |
| pandas                    | Mise en forme des résultats d'agrégation                    |
| MongoDB Atlas             | Instance MongoDB hébergée (cluster gratuit M0)              |
| Streamlit Community Cloud | Hébergement de l'application                                |

Le code est séparé en deux fichiers. `dashboard/mongo_queries.py` contient uniquement les pipelines d'agrégation, qui sont ceux du §5 réécrits en Python. `dashboard/app.py` contient uniquement l'affichage.

### 8.2 Contenu

Les analyses sont réparties en trois onglets : Vue d'ensemble, Genres et temps, Utilisateurs.

Le type de graphique dépend de la donnée : barres horizontales pour les classements, aire pour l'évolution dans le temps, barres verticales pour les heures d'écoute, anneau pour les plateformes, et tableaux pour les résultats qui comportent plusieurs colonnes de chiffres à comparer, comme le taux de réécoute du §5.2.3.

### 8.3 Déploiement

L'application est accessible en ligne :

<https://spotify-listening-analytics-app.streamlit.app/>

Pour être accessible en ligne, l'application doit pouvoir atteindre la base. Le conteneur Docker local ne convenait donc pas : les mêmes 12 000 écoutes et 16 utilisateurs ont été importés sur MongoDB Atlas. Le script `import_mongodb.py` a servi tel quel, l'adresse de connexion étant lue dans une variable d'environnement au lieu d'être écrite dans le code.

Cette adresse est fournie à l'application via les secrets de la plateforme d'hébergement. Elle n'apparaît jamais dans le dépôt Git.

Pour lancer le tableau de bord en local :

```
cp .env.example .env # renseigner MONGO_URI
pip install -r requirements.txt
docker start mongo
streamlit run dashboard/app.py
```

8.4 Capture d'écran

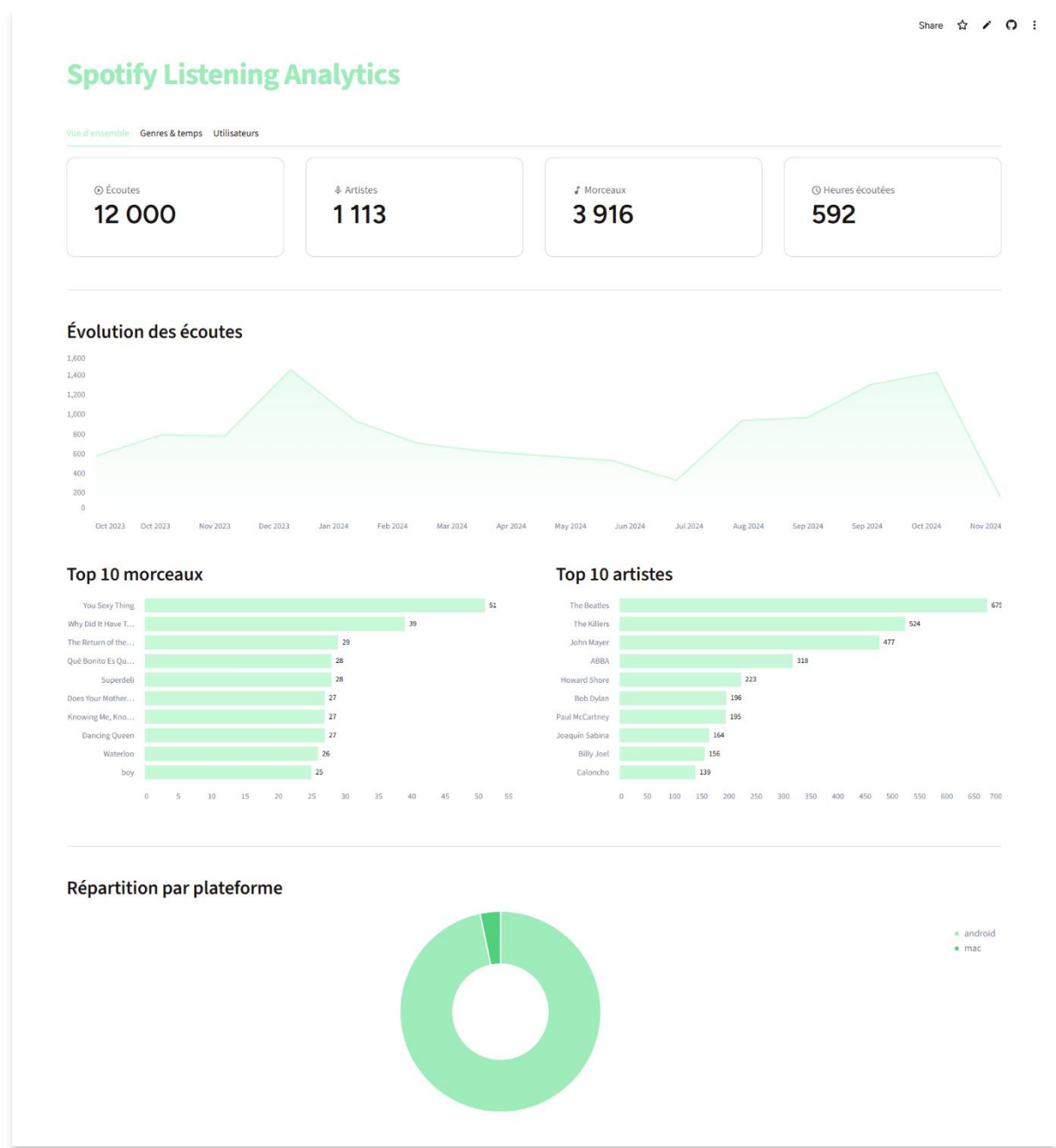


Figure : tableau de bord, onglet « Vue d'ensemble ». Indicateurs globaux et évolution mensuelle des écoutes.

## 9. Analyse critique

### 9.1 Le prix de l'imbrication

Imbriquer le morceau dans chaque écoute simplifie la lecture. Aucune jointure n'est nécessaire pour afficher une écoute ou pour agréger par artiste. En revanche, cela coûte cher en maintenance. Une correction de métadonnées touche N documents. Corriger le genre d'un morceau touche toutes ses écoutes. Le `updateMany` du §4.3 en a modifié 25 d'un coup pour un seul morceau. À l'échelle d'un artiste comme The Killers, la correction toucherait ses 524 écoutes. Un modèle référencé avec une collection morceaux séparée aurait limité cette correction à un seul document, mais il aurait imposé une jointure à chaque lecture.

Dans notre cas, ce coût reste théorique. Une écoute est un événement passé et figé. Ses métadonnées n'ont pas vocation à être corrigées. C'est ce qui rend l'imbrication légitime ici.

### 9.2 L'enrichissement par une source externe

Le genre musical n'existe dans aucune source ouverte sous forme structurée. Spotify le proposait au niveau de l'artiste, mais son API est devenue payante en février 2026. Last.fm n'expose que des tags écrits par les utilisateurs, et ces tags sont bruités. À côté de rock et indie on trouve `seen live`, `60s` ou `american`.

Nous avons donc dû normaliser ces données. Plutôt que d'inventer une liste de genres, nous l'avons dérivée des données elles-mêmes. Les tags les plus fréquents sur nos 1 113 artistes ont été comptés, puis les vrais genres ont été retenus manuellement. Le résultat couvre 96,7 % des écoutes.

Ce détour montre une réalité du métier. La donnée idéale n'existe pas toujours. Une partie du travail consiste à la reconstruire en acceptant ses limites.

### 9.3 La simulation des utilisateurs

C'est la limite la plus importante du projet. Le dataset ne contient qu'une seule personne. Nos 16 utilisateurs sont créés par découpage chronologique de l'échantillon.

La conséquence est directe. Chaque utilisateur n'existe que sur une période d'environ cinq semaines. Les analyses temporelles par utilisateur reflètent donc le découpage et non un comportement réel. Le taux de réécoute du §5.2.3 est particulièrement touché. Un morceau écouté sur une seule période n'a qu'un seul utilisateur distinct, par construction.

Les analyses globales restent valides.

### 9.4 La stratégie d'indexation

Le champ `morceau.artiste` n'a pas été choisi au hasard. Un index n'est utile que sur un champ à forte cardinalité, c'est-à-dire avec beaucoup de valeurs différentes. `morceau.artiste` en compte 1 113. Filtrer dessus élimine presque toute la collection.

Un index sur `shuffle` serait inutile. Ce champ n'a que deux valeurs. Filtrer sur `shuffle: true` retourne encore 74 % des documents et MongoDB ignorerait probablement l'index. L'index sur `morceau.genre` se situe entre les deux. Les genres sont très déséquilibrés, donc il apporte peu sur un genre dominant comme le rock, mais il reste efficace sur les genres rares.

Le gain mesuré est net. MongoDB examinait 12 000 documents avant l'indexation, il n'en examine plus que 524. Le temps d'exécution baisse aussi, mais cette mesure est peu fiable sur un volume aussi faible. Nous ne l'avons donc pas retenue comme indicateur principal.

## 9.5 Les limites de MongoDB observées

Trois contraintes sont apparues pendant le projet.

D'abord l'absence de clé étrangère. Rien n'empêche d'insérer une écoute qui pointe vers un utilisateur inexistant. Ensuite l'absence de suppression en cascade. Il faut gérer soi-même le lien entre `ecoutes` et `utilisateurs` comme le montre le §4.4. Oublier la seconde requête laisserait un profil orphelin. Enfin l'absence de schéma imposé. Cela donne de la souplesse mais laisse la cohérence des documents entièrement au code.

Ces trois points ont un point commun. Ce qu'une base relationnelle garantit, MongoDB le laisse à l'application.